



OBSERVABILITÉ D'INFRASTRUCTURE — TUTO MLOPS

Grafana, Loki & Prometheus

Des métriques chiffrées aux logs détaillés : comprendre, corrélérer et résoudre les incidents en quelques minutes.

Métriques

Logs

LogQL

Alerting

Corrélation

Qu'est-ce que l'observabilité ?



Les métriques — Le « Combien »

Comme ouvrir le gestionnaire de tâches pour surveiller le CPU ou la mémoire vive utilisée. Cela quantifie l'état général du système.



Les logs — Le « Pourquoi »

Comme afficher des variables dans le code pour comprendre pourquoi une fonction bugue. Ils racontent l'histoire détaillée des événements.



Le problème d'échelle : avec des dizaines de machines et d'applications, impossible de se connecter à chaque serveur pour lire les logs un par un.

Pourquoi Loki ? Le choix de la légèreté

Loki, c'est « *le Prometheus des logs* » : un système de centralisation des logs conçu pour s'intégrer parfaitement à Grafana.

Solutions traditionnelles (ELK)

Indexent l'intégralité du texte brut des logs.

Coûte très cher en stockage et en puissance de calcul.



Loki

N'indexe que des labels (nom du serveur, de l'application, du pod...).

Les logs sont regroupés dans des flux très compressés.

Léger · Rapide · Économique

Premier contact avec LogQL

Une requête se lit en deux temps : on sélectionne un flux de logs par ses labels, puis on filtre son contenu.

```
{app="model-serving", env="prod"} |= "error"
```

```
{app="...", env="..."}
```

Sélecteur de labels

Identique à un sélecteur Prometheus : il cible un flux de logs précis parmi tous ceux collectés par Promtail.

```
|= "error"
```

Filtre de contenu

Ne garde que les lignes contenant "error". On peut chaîner `~` (regex), `!=` ou `| json` pour parser des champs.

Pour aller plus loin : `{app="model-serving"} | json | latency_ms > 500` → parse le log en JSON et filtre sur un champ numérique.

Prometheus & Loki : même logique

Loki reprend volontairement le langage de Prometheus — mêmes labels, même fenêtre temporelle — pour rester intuitif.



PromQL

```
rate(http_requests_total  
  {job="model-serving"}[5m])
```

Taux de requêtes par seconde sur les 5 dernières minutes.

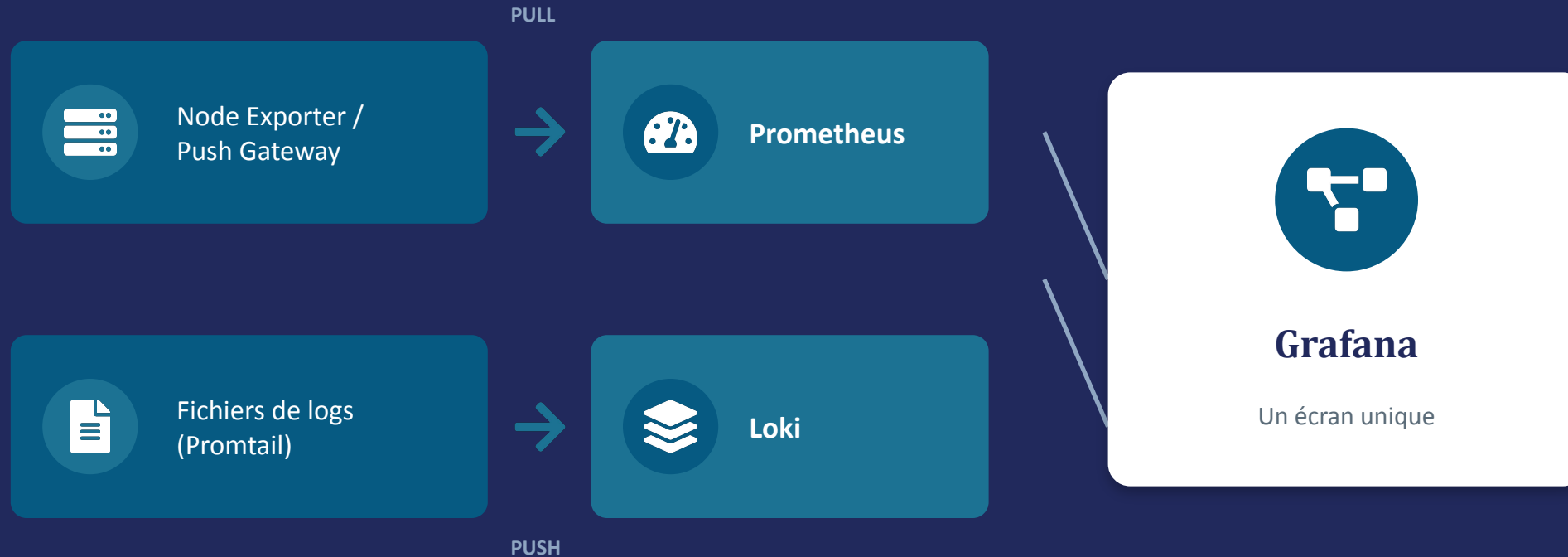


LogQL

```
rate({app="model-serving"}  
  |= "error" [5m])
```

Taux d'apparition de logs d'erreur sur la même fenêtre de 5 minutes.

Comment ça marche ? L'architecture



Le piège de la cardinalité

L'erreur n°1 des débutants avec Loki — et la plus coûteuse en performance.



Chaque combinaison unique de valeurs de labels crée un flux distinct dans Loki. Avec des milliers de valeurs uniques, l'index explose et les performances s'effondrent.



À éviter dans les labels

`user_id`, `request_id`, `trace_id`, `session_id`, adresse IP, timestamp exact...

Ce sont des valeurs à très haute cardinalité : des milliers, voire des millions de combinaisons possibles.



À privilégier

Labels : `app`, `env`, `pod`, `namespace` — faible cardinalité, stables dans le temps.

Détail (`user_id`, `request_id`...) : dans le corps du log, filtrable avec `|=` ou `| json`.

Appliqué au serving de modèles

Ce que ça donne concrètement sur un service d'inférence en production.



Côté Prometheus

- Latence d'inférence (p50 / p95 / p99)
- Taux d'erreurs HTTP 5xx du service
- Utilisation GPU / mémoire par pod
- Nombre de requêtes par modèle/version



Côté Loki

- Exceptions de prédiction (stack trace)
- Version du modèle chargée au démarrage
- Payload rejeté par la validation d'entrée
- Logs applicatifs du pipeline de features

Setup pratique : la stack en local

De quoi suivre ce tuto en pratique — quatre services, un seul fichier.

```
# docker-compose.yml
services:
  prometheus:
    image: prom/prometheus
  loki:
    image: grafana/loki
  promtail:
    image: grafana/promtail
  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
```



En 3 étapes

1. Lancer la stack

```
docker compose up -d
```

2. Ouvrir Grafana

```
http://localhost:3000
```

3. Ajouter les sources

```
Prometheus + Loki en data sources
```

Cas concret : résolution d'un incident

Illustration fil rouge — l'alerte et l'analyse.



1 · L'alerte

Alertmanager détecte que la mémoire d'un pod s'envole anormalement à 3h du matin et notifie l'équipe sur Slack.

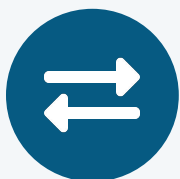


2 · L'analyse dans Grafana

L'ingénieur d'astreinte ouvre son dashboard et constate immédiatement un pic de consommation CPU / mémoire.

Cas concret : résolution d'un incident

Illustration fil rouge — la bascule vers les logs.



3 · La bascule instantanée

Grâce aux labels partagés entre Prometheus et Loki (ex : le pod), un clic sur le graphique ouvre l'onglet Explore, directement sur les logs de l'app, à la seconde du pic.



4 · Filtrage LogQL

Un filtre textuel isole les logs contenant « Error ». La ligne fautive apparaît instantanément : une requête corrompue. Incident résolu en quelques minutes.

Pour aller plus loin

Deux briques complémentaires une fois les bases posées.



Écrire une règle d'alerte

```
- alert: HighInferenceLatency
  expr: histogram_quantile(0.95,
    rate(inference_ms_bucket[5m])) > 500
  for: 5m
  labels: {severity: warning}
```

Alertmanager route ensuite la notification vers Slack, mail ou PagerDuty.



Le 3^e pilier : les traces

Metrics + logs couvrent le « combien » et le « pourquoi ». Il manque le « où » : suivre une requête précise à travers tous les services qu'elle traverse.

Grafana Tempo complète la stack pour le tracing distribué — même principe de corrélation par labels avec Grafana comme point d'entrée unique.

Dashboards communautaires & conclusion



La force de Grafana

Pas besoin de tout créer de zéro : on importe des dashboards JSON prêts à l'emploi, créés par la communauté, pour surveiller CPU, mémoire et réseau en un clin d'œil.



La corrélation, clé de tout

Trouver la cause d'une anomalie chiffrée (métrique) en lisant instantanément l'explication textuelle (log) — au même endroit.

Ressources

Pour approfondir après le tuto.



Documentation Grafana

grafana.com/docs/grafana



Documentation Loki

grafana.com/docs/loki



Documentation Prometheus

prometheus.io/docs



Référence LogQL

grafana.com/docs/loki/latest/query



Référence PromQL

prometheus.io/docs/prometheus/latest/querying



Merci